AI ARENA PLATFORM

Hosting Design

Architecture & Operations Reference · April 2026

Architecture Overview

```
GoDaddy / Cloudflare:
  aiarena.callidity.com          → Railway landing-prod service
  staging.aiarena.callidity.com  → Railway landing-staging service

Railway services (prod):
  landing-prod       ← aiarena.callidity.com          (platform + path-based proxy)
  backend-prod       ← Railway private network only   (no public custom domain)
  xo-prod            ← Railway private network only   (no public custom domain)
  tournament-prod    ← Railway private network only   (no public custom domain)

Railway services (staging):
  landing-staging    ← staging.aiarena.callidity.com  (platform + path-based proxy)
  backend-staging    ← Railway private network only
  xo-staging         ← Railway private network only
  tournament-staging ← Railway private network only
```

URL Structure (prod)

```
aiarena.callidity.com/          → platform (tournaments, rankings, admin)
aiarena.callidity.com/xo/       → XO game (static files proxied from xo-prod)
aiarena.callidity.com/reversi/  → Reversi game (future)
aiarena.callidity.com/api/      → backend API (proxied from backend-prod)
```

URL Structure (staging)

```
staging.aiarena.callidity.com/       → platform
staging.aiarena.callidity.com/xo/    → XO game
staging.aiarena.callidity.com/api/   → backend API
```

Key Principles

- One domain per environment. No wildcards, no subdomains per game.
- The landing service is the single entry point — it proxies all paths to the appropriate Railway private service: `/xo/` → xo service, `/api/` → backend.
- Auth lives on the root domain. Everything is same-origin — sign in once, works everywhere.
- Adding a new game = deploy a Railway service + one proxy rule in landing. No DNS changes, no Railway Pro plan.
- All backend and game services use Railway private networking — not reachable from the public internet.
- Each game is an independent Railway service with its own deploy lifecycle, resource allocation, and failure isolation.

Monorepo Structure

Railway Service	Source directory	Dockerfile
`landing`	`landing/`	`landing/Dockerfile`
`backend`	`backend/`	`backend/Dockerfile`
`xo`	`frontend/`	`frontend/Dockerfile`
`tournament`	`tournament/`	`tournament/Dockerfile`

Railway's watch paths ensure only the affected service redeploys on a commit:

- Commit touches `frontend/` → only XO service redeploys
- Commit touches `landing/` → only landing redeploys
- Commit touches `backend/` → only backend redeploys
- Commit touches `packages/` → all services redeploy (shared code changed)

Branch → Environment Mapping

Branch	Environment	Domain
`main`	Production	`aiarena.callidity.com`
`staging`	Staging	`staging.aiarena.callidity.com`
`dev`	Local only	`localhost`

Promotion flow: `dev` → `staging` → `main`

GoDaddy DNS Records

Root domain in GoDaddy is `callidity.com`.

Host	Type	Points To
`aiarena`	CNAME	Railway landing-prod CNAME target
`staging.aiarena`	CNAME	Railway landing-staging CNAME target

2 records total. Never changes regardless of how many games are added.

> **Recommended:** Point GoDaddy nameservers to Cloudflare (free tier) for faster global DNS resolution, DDoS protection, WAF, and caching rules. With Cloudflare, static game assets (`/xo/*`, `/reversi/*`) can be cached at the edge, eliminating proxy load entirely. See the Cloudflare section below.

Railway — Custom Domain Config

Only the landing services need custom domains. All other services use Railway private networking.

Railway Service	Custom Domain
`landing-prod`	`aiarena.callidity.com`
`landing-staging`	`staging.aiarena.callidity.com`

> No Railway Pro plan required — wildcard domains are not used. Enable Railway private networking for backend, xo, and tournament services so they are only reachable by other services within the same Railway environment.

Landing Service — Path-Based Proxy

The landing service is the single entry point for all traffic. It proxies both game static files and API calls to the appropriate private Railway service.

```
const PROXY_RULES = {
  '/api':     process.env.BACKEND_PRIVATE_URL,   // e.g. http://backend.railway.internal:3000
  '/xo':      process.env.XO_PRIVATE_URL,        // e.g. http://xo.railway.internal:80
  '/reversi': process.env.REVERSI_PRIVATE_URL,   // future
}
```

> **Port note:** Railway private URLs use the format `http://<service>.railway.internal` or `http://<service>.railway.internal:PORT` if the internal port is not 80. Check each service's Railway settings and include the port explicitly.

What the Proxy Handles

- `/api/*` → backend (all API calls, OAuth callbacks, WebSocket upgrades)
- `/xo/*` → XO static file server (HTML, JS, CSS bundles)
- `/reversi/*` → Reversi static file server (future)

Required Proxy Headers

```
'X-Forwarded-For':   req.headers['x-forwarded-for'] || req.socket.remoteAddress,
'X-Forwarded-Host':  req.headers.host,
'X-Forwarded-Proto': 'https',
```

Proxy Robustness

- Set sensible timeouts (connect: 5s, response: 30s) so a slow upstream doesn't hang the landing service
- Use connection pooling (`keepAlive: true` in http-proxy)
- Consider a circuit breaker so a crashed game service returns a clean error page rather than hanging connections

WebSocket Proxying

WebSocket connections go through the landing proxy. The proxy must handle the HTTP → WebSocket upgrade:

```
// Example using http-proxy
proxy.ws(req, socket, head, { target: BACKEND_PRIVATE_URL })
```

SPA Fallback Routing

The XO app is a React SPA — only `index.html` exists on the static file server. Any unmatched path under `/xo/*` must serve `/xo/index.html` so React Router handles client-side navigation. Without this, any page refresh or direct URL returns a 404.

```
// If the XO service returns 404, serve its index.html instead
if (res.statusCode === 404 && req.path.startsWith('/xo')) {
  return proxyTo(XO_PRIVATE_URL, '/xo/index.html')
}
```

Required Code Changes — XO Frontend

Two changes are required in the XO Vite app before this design can work. These only apply to production builds; local dev is unaffected.

1. Vite Base Path

The built `index.html` must reference assets relative to `/xo/`, not `/`. Without this, all JS/CSS bundles 404 immediately.

```
// frontend/vite.config.js
export default defineConfig({
  base: process.env.VITE_BASE_PATH ?? '/',
  // ...
})
```

Set `VITE_BASE_PATH=/xo/` in the Railway xo-prod and xo-staging environment variables at build time.

2. React Router Basename

React Router must know it is mounted at `/xo/` so navigation resolves correctly.

```
// frontend/src/main.jsx
<BrowserRouter basename={import.meta.env.VITE_BASE_PATH ?? '/'}>
  <App />
</BrowserRouter>
```

Cloudflare Setup (Recommended)

Point GoDaddy nameservers to Cloudflare for the `callidity.com` zone. The free tier provides DDoS protection, WAF, and caching.

DNS Records in Cloudflare

Name	Type	Content	Proxy
`aiarena.callidity.com`	CNAME	Railway landing-prod target	Orange cloud (proxied)
`staging.aiarena.callidity.com`	CNAME	Railway landing-staging target	Grey cloud (DNS only)

> Proxy staging through DNS only (grey cloud) to avoid Cloudflare cache interfering with testing.

Cloudflare Cache Rules (prod only)

```
If URI path matches /xo/*    → Cache everything, TTL 1 day
If URI path matches /api/*   → Bypass cache
```

Environment Variables

Landing Service

Variable	Prod value	Staging value
`BACKEND_PRIVATE_URL`	`http://backend.railway.internal:3000`	`http://backend-staging.railway.internal:3000`
`XO_PRIVATE_URL`	`http://xo.railway.internal:80`	`http://xo-staging.railway.internal:80`
`TOURNAMENT_PRIVATE_URL`	`http://tournament.railway.internal:3001`	`http://tournament-staging.railway.internal:3001`

XO Service (build-time)

Variable	Value
`VITE_BASE_PATH`	`/xo/`

Backend Service

Variable	Prod value	Staging value
`BETTER_AUTH_URL`	`https://aiarena.callidity.com`	`https://staging.aiarena.callidity.com`
`FRONTEND_URL`	`https://aiarena.callidity.com`	`https://staging.aiarena.callidity.com`
`NODE_ENV`	`production`	`production`
`GOOGLE_CLIENT_ID`	shared across environments	
`GOOGLE_CLIENT_SECRET`	shared across environments	
`APPLE_CLIENT_ID`	`com.callidity.aiarena.web`	
`APPLE_PRIVATE_KEY`	contents of .p8 file	
`APPLE_KEY_ID`	10-char key ID	
`APPLE_TEAM_ID`	10-char team ID	

> No `BETTER_AUTH_COOKIE_DOMAIN` needed — everything is same-origin. No CORS configuration needed — all browser requests arrive via the landing proxy. The backend never receives direct browser requests.

Google OAuth (Google Cloud Console)

AUTHORIZED JAVASCRIPT ORIGINS

```
https://aiarena.callidity.com
https://staging.aiarena.callidity.com
http://localhost:5174
```

AUTHORIZED REDIRECT URIS

```
https://aiarena.callidity.com/api/auth/callback/google
https://staging.aiarena.callidity.com/api/auth/callback/google
http://localhost:5174/api/auth/callback/google
```

Apple OAuth (Apple Developer Console)

Apple requires domain ownership verification and only works on live HTTPS domains. Configure this after the custom domains are live on Railway.

Step 1 — App ID

In *Certificates, Identifiers & Profiles → Identifiers*, confirm **Sign In with Apple** is enabled on your App ID.

Step 2 — Service ID

- **Identifier:** `com.callidity.aiarena.web`
- Enable **Sign In with Apple** → Configure:
  - **Domains:** `aiarena.callidity.com`, `staging.aiarena.callidity.com`
  - **Return URLs:** `https://aiarena.callidity.com/api/auth/callback/apple` and staging equivalent

Step 3 — Domain Verification File

Deploy the file Apple provides to:

```
landing/public/.well-known/apple-developer-domain-association.txt
```

The static middleware serves `public/` automatically — no code changes needed.

Step 4 — Private Key

Create a key in *Keys* with **Sign In with Apple** enabled. Download the `.p8` file — **Apple only lets you download it once.** Store it securely and set as `APPLE_PRIVATE_KEY` in Railway.

Local Dev

All services run via `docker compose up`:

Service	Port	Purpose
postgres	5432	Shared database
redis	6379	Pub/sub + caching
backend	3000	Auth, API, WebSockets
tournament	3001	Tournament CRUD and matches
frontend (xo)	5173	XO game UI
landing	5174	Platform UI + proxy

In local dev the two UIs run on separate ports for simplicity. The path-based proxy is only active in the production landing build. `VITE_BASE_PATH` is not set in dev — the XO app runs at `/` with no adjustment needed.

Adding a New Game

1. Build the game and add it to the monorepo (e.g. `reversi/`)
2. Add a `reversi/Dockerfile`
3. Create a Railway service with watch paths `reversi/**,packages/**`
4. Enable Railway private networking on the new service
5. Add `REVERSI_PRIVATE_URL` env var to landing-prod and landing-staging
6. Add one proxy rule to landing: `'/reversi': process.env.REVERSI_PRIVATE_URL`
7. Set `VITE_BASE_PATH=/reversi/` on the Reversi Railway service
8. Add SPA fallback rule for `/reversi/*` in the landing proxy
9. Redeploy landing

No DNS changes. No GoDaddy. No Cloudflare changes. No Railway custom domain config. No Railway Pro plan.

Risks and Mitigations

Risk	Likelihood	Impact	Mitigation
Landing service outage (SPOF)	Medium	High	Enable Railway replicas (2–3) on landing-prod; add health-check monitoring and alerts
Brief downtime during landing redeploys	Low	Medium	Acceptable at current scale; use Railway staged deploys if it becomes a concern
Proxy misconfiguration (WebSocket, SPA fallback)	Low	High	Thorough end-to-end testing on staging before prod cutover, including deep links and auth flows
Future high traffic overwhelming proxy	Low	Medium	Add Cloudflare — caches static `/xo/*` and `/reversi/*` at the edge, eliminating proxy load

Order of Operations (First Production Deploy)

1. Deploy all Railway services — enable private networking on backend, xo, tournament
2. Note each service's private Railway URL and port
3. Set env vars on each service (see sections above)
4. Add custom domains to `landing-prod` and `landing-staging` in Railway dashboard
5. Point GoDaddy nameservers to Cloudflare; add the 2 CNAME records in Cloudflare
6. Wait for DNS propagation (verify with `dig aiarena.callidity.com`)
7. Confirm Railway shows domains verified with valid SSL
8. Enable Railway replicas (2–3) on landing-prod
9. Deploy Apple domain verification file to landing, redeploy
10. Configure Apple Service ID (domain must be live with HTTPS first)
11. Configure Google OAuth origins and redirect URIs
12. Redeploy backend with final env vars
13. Set up basic monitoring (Railway metrics + Sentry or Railway log alerts)
14. Test full auth and game flow end-to-end on staging, including Apple Sign In on a real device
15. Promote to prod; verify sign in, `/xo/` navigation, WebSocket connection, and a complete game session

When Traffic Grows

These steps are not needed now but are straightforward when the time comes:

- **Enable Cloudflare caching rules** for `/xo/*` and `/reversi/*` — eliminates almost all proxy load for static assets
- **Add Railway autoscaling** on landing-prod — handles traffic spikes without manual intervention

- **Split game backends** — if a specific game's backend becomes a bottleneck, extract it to its own Railway service; the landing proxy updates one env var

---

Appendix — Architecture Tradeoffs

Chosen Approach: Path-Based Routing via Landing Proxy

Pros	Cons
2 DNS records total, never changes	Landing is a single point of failure (mitigated by replicas)
No Railway Pro plan needed	Redeploying landing briefly disrupts the platform
No wildcard domains	Game URLs are paths not subdomains (cosmetic)
Same-origin — no CORS, no cross-site auth complexity	
Sign in once, works everywhere automatically	
Backend and game services are fully private	
Adding a game = one proxy rule + one env var	
Each game still an independent Railway service	
Static files browser-cached — proxy load minimal	
Cloudflare CDN eliminates proxy load when needed	

Alternatives Considered

OPTION A — GAME SUBDOMAINS WITH GATEWAY SERVICE

```
aiarena.callidity.com         → platform
xo.aiarena.callidity.com      → gateway → XO Railway service
reversi.aiarena.callidity.com → gateway → Reversi Railway service
```

Pros	Cons
Each game on its own subdomain	Requires Railway Pro plan (wildcard custom domains)
Gateway failure doesn't take down platform	4+ DNS records, grows with each game
	Extra gateway service to build and maintain
	Cross-site auth requires `BETTER_AUTH_COOKIE_DOMAIN`

OPTION B — GAME SUBDOMAINS, DIRECT CUSTOM DOMAINS

Pros	Cons
No gateway service	One new DNS record + Railway custom domain per game
Each game fully isolated	Requires Railway Pro plan per custom domain
	Same cross-site auth complexity as Option A
	Manual work every time a game is added

OPTION C — RAILWAY INTERNAL URLS, NO CUSTOM GAME DOMAINS

Pros	Cons
Zero DNS setup for games	Cross-origin auth — cookies don't carry
No Railway Pro plan	Requires server-side token denylist for sign-out
	Railway `.up.railway.app` URLs are publicly accessible
	Ugly, unstable URLs

Why the Chosen Approach Wins

At current scale the proxy load is negligible — only static files, cached after first visit. The auth story is the simplest possible (same origin, no extra config). The operational story is the simplest possible (2 DNS records, set once). Backend and game services are fully private. The only real risk is landing being a single point of failure, mitigated cheaply with Railway replicas. The database and backend are far more likely bottlenecks in practice.